ĐẶC TẢ THIẾT KẾ gemm_dma_engine

Module của `gemm_accel_top` · instance `u_dma`

Mục lục

1 Tổng quan **2**

1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 3
1.4 Tín hiệu AXI . . . 6

2 Đặc tả thanh ghi **8**

3 Mô tả chức năng **8**

3.1 Tổ chức chức năng . . . 8
3.2 `gemm_dma_agen`: địa chỉ và burst . . . 9
3.3 `gemm_dma_rd`: kênh đọc . . . 11
3.4 `gemm_dma_wr`: kênh ghi . . . 12
3.5 Reset, hiện thực, kiểm chứng . . . 14

1 Tổng quan

1.1 Tính năng chính

DMA chép dữ liệu giữa DRAM và scratchpad theo lệnh của Main FSM. **Kênh đọc** chạy FETCH, LOAD_A, LOAD_B, LOAD_BIAS: đọc DRAM bằng burst AXI và ghi vào SRAM (hoặc trả 16 word descriptor cho FSM). **Kênh ghi** chạy STORE_C: đọc buffer C rồi ghi DRAM. Hai kênh độc lập hoàn toàn, mỗi kênh tối đa một burst outstanding. Mỗi lệnh là một bản sao 2D: `rows` hàng, mỗi hàng `rwords` word, các hàng cách nhau `stride` byte. Module là bên duy nhất của `gemm_accel_top` phát giao dịch trên cổng AXI4-Full master.

Chức năng

- Với lệnh (base, rows, rwords, stride), word thứ n ($0 \le n < \text{rows} \cdot \text{rwords}$), r = n / rwords, w = n mod rwords, nằm ở địa chỉ DRAM base + r · stride + 4w và ở địa chỉ scratchpad theo các quy tắc địa chỉ ở mục 3.2. Mỗi word được chép đúng một lần, theo thứ tự n tăng dần
- Mỗi lệnh kết thúc bằng đúng một xung `o_*_done` hoặc `o_*_error`. Lỗi: `rresp` hoặc `bresp` $\neq$ OKAY, `rlast` sai vị trí, hoặc tham số lệnh không hợp lệ
- AXI4 master 32 bit: `arsize` = `awsize` = 3'b010 (4 byte), burst INCR, `wstrb` = 4'b1111, `bready` = 1 cố định. Tối đa 1 burst đọc và 1 burst ghi outstanding
- Burst không bao giờ vượt biên 4 KiB. Số beat của mỗi burst là giá trị nhỏ nhất trong ba giới hạn: số word còn lại của đoạn, `MAX_BURST_BEATS`, và $1024 - a[11{:}2]$
- Tham số lệnh không hợp lệ (`rows` = 0, `rwords` = 0, `stride` không là bội của 4, `base` không căn 4 byte) làm lệnh kết thúc ngay bằng `o_*_error`, không phát giao dịch bus và không truy cập SRAM. Đây là danh sách đóng: các ràng buộc còn lại trong bảng port (`rows` $\le$ 32, `rwords` $\le$ 32, `stride` $\ge 4 \cdot$ `rwords`) **không** được DMA kiểm mà do Main FSM bảo đảm (descriptor ngoài miền M, N, K đã bị loại ở bước kiểm tra; stride 0 được FSM thay bằng đúng độ dài hàng). Lệnh vi phạm các ràng buộc đó có hành vi không xác định
- `i_rd_start` đến khi kênh đọc đang bận thì bị bỏ qua; `i_wr_start` với kênh ghi tương tự. Abort không tự tạo lỗi
- Dữ liệu của beat lỗi vẫn được ghi vào SRAM; FIFO ghi được xóa sau lỗi hoặc abort. FSM không dùng kết quả của lệnh lỗi
- DMA không đảo byte, không đổi dữ liệu; không biết M, N, K hay ý nghĩa của buffer
- Kênh đọc và kênh ghi được phép có burst AXI cùng lúc, vì chúng không dùng chung tín hiệu AXI nào

1.2 Tham số

Bảng 1.1: Tham số

Tham số	Mặc định	Ý nghĩa
`MAX_BURST_BEATS`	16	Số beat tối đa mỗi burst, 1 … 256; baseline chốt 16 (D-36). Nối từ tham số cùng tên của `gemm_accel_top`. Giá trị ngoài 1 … 256 bị chặn lúc elaboration
`WFIFO_DEPTH`	4	localparam: số word của FIFO dữ liệu ghi. 4 đủ cho 1 beat W mỗi chu kỳ vì SRAM trả dữ liệu sau đúng 1 chu kỳ

1.3 Tín hiệu giao tiếp

Bảng 1.2: Tín hiệu nội bộ của `gemm_dma_engine`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Lệnh kênh đọc			
`i_rd_start`	1	Input	Xung một chu kỳ yêu cầu bắt đầu lệnh đọc DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có i_rd_start = 1.
`i_rd_dst`	2	Input	Chọn đích của lệnh đọc. Giá trị 0: buffer A. Giá trị 1: buffer B. Giá trị 2: buffer BIAS. Giá trị 3: descriptor trả về Main FSM. Giá trị được chốt cùng xung i_rd_start.
`i_rd_base`	32	Input	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh đọc DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung i_rd_start.
`i_rd_rows`	6	Input	Số hàng cần truyền trong lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_rd_start.
`i_rd_rwords`	6	Input	Số word 32 bit cần truyền trong mỗi hàng của lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_rd_start.
`i_rd_stride`	16	Input	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung i_rd_start.
`i_rd_bank`	1	Input	Chọn bank đích của buffer A, B hoặc BIAS khi nạp dữ liệu; không dùng cho FETCH. Giá trị được chốt cùng xung i_rd_start.
`i_rd_abort`	1	Input	Yêu cầu dừng lệnh đọc DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`o_rd_busy`	1	Output	Báo kênh đọc DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận i_rd_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.

Bảng 1.2: Tín hiệu nội bộ của `gemm_dma_engine` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_rd_done`	1	Output	Xung một chu kỳ báo lệnh đọc DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`o_rd_error`	1	Output	Xung một chu kỳ báo lệnh đọc DRAM kết thúc có lỗi. Lỗi gồm phản hồi R khác OKAY, RLAST sai vị trí hoặc tham số lệnh không hợp lệ (khi đó phát ở chu kỳ ngay sau start, không truy cập bus và SRAM). Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
`o_desc_valid`	1	Output	Báo có một word descriptor hợp lệ trong chu kỳ này. Chỉ dùng với lệnh FETCH; bên nhận lấy dữ liệu khi valid = 1, không có ready.
`o_desc_idx`	4	Output	Chỉ số word trong descriptor, từ 0 đến 15. Hợp lệ cùng chu kỳ với o_desc_valid.
`o_desc_data`	32	Output	Nội dung 32 bit của word descriptor có chỉ số o_desc_idx. Hợp lệ cùng chu kỳ với o_desc_valid; dữ liệu của FETCH bị lỗi không được dùng để thực thi job.
Lệnh kênh ghi			
`i_wr_start`	1	Input	Xung một chu kỳ yêu cầu bắt đầu lệnh ghi DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có i_wr_start = 1.
`i_wr_fmt`	1	Input	Chọn định dạng dữ liệu trong buffer C. Giá trị 0: INT32, mỗi phần tử chiếm một word. Giá trị 1: INT8, bốn phần tử trong một word. Giá trị được chốt cùng xung i_wr_start.
`i_wr_base`	32	Input	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh ghi DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung i_wr_start.
`i_wr_rows`	6	Input	Số hàng cần truyền trong lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_wr_start.
`i_wr_rwords`	6	Input	Số word 32 bit cần truyền trong mỗi hàng của lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_wr_start.
`i_wr_stride`	16	Input	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung i_wr_start.
`i_wr_bank`	1	Input	Chọn bank nguồn của buffer C khi thực hiện STORE_C. Giá trị được chốt cùng xung i_wr_start.
`i_wr_abort`	1	Input	Yêu cầu dừng lệnh ghi DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.

Bảng 1.2: Tín hiệu nội bộ của `gemm_dma_engine` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_wr_busy`	1	Output	Báo kênh ghi DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận i_wr_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`o_wr_done`	1	Output	Xung một chu kỳ báo lệnh ghi DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`o_wr_error`	1	Output	Xung một chu kỳ báo lệnh ghi DRAM kết thúc có lỗi. Lỗi gồm phản hồi B khác OKAY hoặc tham số lệnh không hợp lệ (khi đó phát ở chu kỳ ngay sau start, không truy cập bus và SRAM). Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
Cổng ghi scratchpad (A, B, BIAS)			
`o_spm_wr_valid`	1	Output	Yêu cầu ghi một word 32 bit vào scratchpad. Ghi được nhận khi valid và ready cùng bằng 1 tại cạnh lên.
`o_spm_wr_buf`	2	Output	Chọn buffer đích cho word ghi. Giá trị 0: A. Giá trị 1: B. Giá trị 2: BIAS; giá trị 3 không hợp lệ.
`o_spm_wr_bank`	1	Output	Chọn bank đích cho word ghi. Hợp lệ cùng yêu cầu ghi; nhận giá trị 0 hoặc 1.
`o_spm_wr_addr`	10	Output	Địa chỉ word logic trong buffer đích. A/B dùng miền 0 đến 255; BIAS dùng miền 0 đến 31. Hợp lệ cùng yêu cầu ghi.
`o_spm_wr_data`	32	Output	Word dữ liệu 32 bit cần ghi vào scratchpad. Hợp lệ cùng yêu cầu ghi; DMA giữ nguyên thứ tự byte của dữ liệu DRAM.
`i_spm_wr_ready`	1	Input	Báo scratchpad có thể nhận word ghi. SRAM không có backpressure nên luôn bằng 1; giao dịch vẫn được xác định bởi valid và ready.
Cổng đọc scratchpad (C)			
`o_spm_rd_valid`	1	Output	Yêu cầu đọc một word từ buffer C. Yêu cầu được nhận tại cạnh lên khi valid và ready cùng bằng 1.
`o_spm_rd_bank`	1	Output	Chọn bank nguồn của buffer C. Hợp lệ cùng yêu cầu đọc; nhận giá trị 0 hoặc 1.
`o_spm_rd_addr`	10	Output	Địa chỉ word logic cần đọc trong buffer C. Hợp lệ cùng yêu cầu đọc; miền địa chỉ phụ thuộc bố cục INT32 hoặc INT8 của C.
`i_spm_rd_ready`	1	Input	Báo scratchpad có thể nhận yêu cầu đọc C. SRAM không có backpressure nên luôn bằng 1.
`i_spm_rd_rvalid`	1	Input	Báo word đọc trả về hợp lệ. Xuất hiện đúng một chu kỳ sau yêu cầu được nhận; mỗi yêu cầu có đúng một phản hồi, theo thứ tự.
`i_spm_rd_rdata`	32	Input	Word dữ liệu 32 bit đọc từ buffer C. Bên nhận lấy dữ liệu khi read rvalid = 1; không có ready cho kênh trả dữ liệu.

Giao tiếp

- `i_rd_start` chỉ đến khi `o_rd_busy` = 0. DMA chốt mọi tham số ở chu kỳ đó; FSM được đổi chúng ngay sau. Nếu `i_rd_start` đến khi `o_rd_busy` = 1 thì bị bỏ qua; tương tự `i_wr_start` với `o_wr_busy`
- `i_rd_abort`, `i_wr_abort` được chốt thành cờ `abort_seen` trong suốt lệnh; xóa khi lệnh kết thúc
- `m_axi_arsize` = `m_axi_awsize` = 3'b010; `m_axi_arburst` = `m_axi_awburst` = 2'b01; `m_axi_wstrb` = 4'b1111; `m_axi_bready` = 1
- Mọi port `m_axi_*` ra lấy từ thanh ghi, trừ `m_axi_rready` (= trạng thái RD_DATA $\wedge$ `i_spm_wr_ready`); không có đường tổ hợp từ `m_axi_*` vào tới `m_axi_*` ra
- `o_spm_wr_valid` = `m_axi_rvalid` $\wedge$ `m_axi_rready` $\wedge$ (beat nằm trong burst) $\wedge$ (dst $\neq$ 3); `o_spm_wr_data` = `m_axi_rdata`. Đường tổ hợp này chỉ đi vào SRAM, được phép
- Tương tự với FETCH: `o_desc_valid` = `m_axi_rvalid` $\wedge$ `m_axi_rready` $\wedge$ (beat nằm trong burst) $\wedge$ (dst = 3); `o_desc_data` = `m_axi_rdata`; `o_desc_idx` lấy từ bộ đếm word (thanh ghi). Đường tổ hợp này chỉ đi vào bộ đệm descriptor của Main FSM (kết thúc ở thanh ghi), được phép; cùng nhóm với `o_spm_wr_*`
- `i_spm_wr_ready`, `i_spm_rd_ready` luôn bằng 1 (SRAM không có backpressure); DMA vẫn dùng quy tắc valid $\wedge$ ready để xác định giao dịch

1.4 Tín hiệu AXI

Bảng 1.3: Tín hiệu AXI

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`m_axi_araddr`	32	Output	Địa chỉ byte đầu tiên của burst đọc. Địa chỉ căn 4 byte. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
`m_axi_arlen`	8	Output	Số beat của burst đọc trừ 1. Hợp lệ cùng ARVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
`m_axi_arsize`	3	Output	Kích thước mỗi beat của burst đọc. Luôn bằng 3'b010, tương ứng 4 byte.
`m_axi_arburst`	2	Output	Kiểu burst đọc. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
`m_axi_arvalid`	1	Output	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
`m_axi_arready`	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
`m_axi_rdata`	32	Input	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
`m_axi_rresp`	2	Input	Mã phản hồi beat đọc. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng RVALID.
`m_axi_rlast`	1	Input	Đánh dấu beat cuối burst đọc khi RVALID = 1. DMA so sánh vị trí với ARLEN + 1; đến sớm hoặc muộn đều là lỗi.
`m_axi_rvalid`	1	Input	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.

Bảng 1.3: Tín hiệu AXI (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_rready	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.
m_axi_awaddr	32	Output	Địa chỉ byte đầu tiên của burst ghi. Địa chỉ căn 4 byte. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
m_axi_awlen	8	Output	Số beat của burst ghi trừ 1. Hợp lệ cùng AWVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
m_axi_awsize	3	Output	Kích thước mỗi beat của burst ghi. Luôn bằng 3'b010, tương ứng 4 byte.
m_axi_awburst	2	Output	Kiểu burst ghi. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
m_axi_awvalid	1	Output	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
m_axi_awready	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
m_axi_wdata	32	Output	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
m_axi_wstrb	4	Output	Mặt nạ byte ghi. Luôn bằng 4'b1111 vì DMA ghi nguyên word 32 bit.
m_axi_wlast	1	Output	Đánh dấu beat cuối burst ghi khi WVALID = 1. Bằng 1 ở đúng beat thứ AWLEN + 1 và giữ nguyên trong lúc chờ WREADY.
m_axi_wvalid	1	Output	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
m_axi_wready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
m_axi_bresp	2	Input	Mã phản hồi burst ghi. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng BVALID.
m_axi_bvalid	1	Input	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
m_axi_bready	1	Output	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1. DMA luôn giữ BREADY = 1.

2 Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Toàn bộ thanh ghi MMIO nằm trong `gemm_register_file`, xem đặc tả hệ thống baseline. Cấu hình lệnh được nhận qua các tín hiệu đầu vào `i_rd_*`, `i_wr_*` từ Main FSM; trạng thái nội bộ, điều kiện chốt và reset được mô tả ở phần Mô tả chức năng.

3 Mô tả chức năng

3.1 Tổ chức chức năng

Bảng 3.1: Module con

Module	Instance	Vai trò
`gemm_dma_agen`	`u_agen_rd`, `u_agen_wr`	Chia lệnh thành đoạn liên tục, cắt burst, đếm (r, w) và sinh địa chỉ scratchpad
`gemm_dma_rd`	`u_rd`	Kênh đọc: AR, R; ghi SRAM hoặc xuất descriptor; lỗi, abort
`gemm_dma_wr`	`u_wr`	Kênh ghi: đọc SRAM theo credit, AW, W, B; lỗi, abort
`gemm_dma_wfifo`	`u_wfifo`	FIFO đồng bộ `WFIFO_DEPTH` × 32, đầu ra có thanh ghi (head)

- Đường dữ liệu đọc: AR → R → (`o_spm_wr_*` hoặc `o_desc_*`). Đường dữ liệu ghi: `o_spm_rd_*` → (1 chu kỳ) `i_spm_rd_rvalid/i_spm_rd_rdata` → `u_wfifo` → kênh W; kênh AW chạy song song; sau cùng là kênh B
- `gemm_dma_rd` và `gemm_dma_wr` là hai FSM độc lập; các cổng của chúng chính là phần cổng lệnh kênh đọc/ghi, cổng scratchpad và cổng AXI tương ứng trong bảng tín hiệu ở mục 1.3, nên không lặp lại bảng riêng
- `u_agen_rd` phục vụ `gemm_dma_rd` với `i_wide` = (dst = 2); `u_agen_wr` phục vụ `gemm_dma_wr` với `i_wide` = (fmt = 0). Với dst = 3 (FETCH) `i_wide` không được dùng

Bảng 3.2: Lệnh DMA theo pha (giá trị do Main FSM cấp)

Pha	dst / fmt	rows	rwords	stride	Bank
FETCH	dst = 3	1	16	64	0 (không dùng)
LOAD_BIAS	dst = 2	1	N	4N	bank BIAS đích
LOAD_A	dst = 0	M	K/4	`lda`	bank A đích
LOAD_B	dst = 1	K	N/4	`ldb`	bank B đích
STORE_C	fmt = POST_EN	M	POST_EN ? N/4 : N	`ldc`	bank C nguồn

QUYẾT ĐỊNH BASELINE

D-31 (FETCH). Lệnh FETCH là `o_rd_dst` = 3, rows = 1, rwords = 16, stride = 64, bank 0, một burst INCR 16 beat (khi `MAX_BURST_BEATS` = 16; với giá trị nhỏ hơn, burst được cắt theo quy tắc chung ở mục 3.2). 16 word trả về qua `o_desc_valid`, `o_desc_idx[3:0]`, `o_desc_data`; descriptor không bao giờ đi vào scratchpad. Main FSM đảm bảo `base` căn 64 byte, nên burst FETCH không vượt biên 4 KiB.

3.2 gemm_dma_agen: địa chỉ và burst

Bảng 3.3: Tín hiệu giao tiếp của gemm_dma_agen

Tên tín hiệu	Độ rộng	Hướng	Mô tả
i_load	1	Input	Xung chốt các tham số lệnh mới tại cạnh lên. Được phát cùng chu kỳ start của kênh DMA tương ứng.
i_base	32	Input	Địa chỉ byte đầu lệnh trong DRAM, căn 4 byte. Chốt khi i_load = 1.
i_rows	6	Input	Số hàng cần truyền, từ 1 đến 32. Chốt khi i_load = 1.
i_rwords	6	Input	Số word 32 bit của mỗi hàng, từ 1 đến 32. Chốt khi i_load = 1.
i_stride	16	Input	Khoảng cách byte giữa đầu hai hàng. Là bội của 4, không nhỏ hơn 4 lần i_rwords; chốt khi i_load = 1.
i_wide	1	Input	Chọn bố cục địa chỉ scratchpad. Giá trị 1: ghép r[4:0] với w[4:0]; 0: ghép r[4:0] với w[2:0]. Chốt cùng lệnh.
o_has_burst	1	Output	Bằng 1 khi còn burst chưa phát của lệnh hiện tại. Khi bằng 0, kênh DMA không được khởi tạo burst tiếp theo.
o_burst_addr	32	Output	Địa chỉ byte đầu burst kế tiếp trong DRAM. Là đầu ra thanh ghi, hợp lệ từ chu kỳ sau i_load hoặc i_burst_next khi còn burst.
o_burst_beats	9	Output	Số beat của burst kế tiếp, từ 1 đến 256. Là đầu ra thanh ghi, đi cùng o_burst_addr; đã xét giới hạn độ dài và biên 4 KiB.
i_burst_next	1	Input	Xung báo burst hiện tại đã được lấy để phát. Cập nhật mô tả burst tiếp theo ở chu kỳ sau; không tự tăng bộ đếm word.
i_word_next	1	Input	Xung báo một word đã được chuyển. Tăng chỉ số word w; khi hết hàng, đưa w về 0 và tăng hàng r.
o_spm_addr	10	Output	Địa chỉ word logic của phần tử đang truyền trong scratchpad. Được tạo từ chỉ số hàng r và word w theo i_wide.
o_word_idx	4	Output	Bốn bit thấp của số thứ tự word đang truyền. Với FETCH, đây là chỉ số word descriptor từ 0 đến 15.

Bảng 3.4: Thanh ghi nội bộ của gemm_dma_agen

Thanh ghi	Rộng	Vai trò
seg_base	32	Địa chỉ byte đầu đoạn hiện tại. i_load: ← i_base. Mỗi lần chuyển sang đoạn kế: ← seg_base + stride
cur_addr	32	Địa chỉ byte đầu burst kế; chính là o_burst_addr
seg_left	11	Số word còn lại của đoạn hiện tại (tối đa 1024)
seg_len	11	Độ dài mỗi đoạn: rows · rwords khi lệnh liên tục, rwords khi có hở
segs_left	6	Số đoạn còn lại, kể cả đoạn hiện tại
o_burst_beats	9	Số beat burst kế (1 … 256)
o_has_burst	1	Còn burst chưa phát
r, w	5, 5	Bộ đếm hàng và word trong hàng cho địa chỉ scratchpad; độc lập với burst

Bảng 3.4: Thanh ghi nội bộ của `gemm_dma_agen` (tiếp theo)

Thanh ghi	Rộng	Vai trò

Bảng 3.5: Địa chỉ scratchpad theo đích

Lệnh	i_wide	Địa chỉ word n (r = hàng, w = word trong hàng)
LOAD_A, LOAD_B	0	{r[4:0], w[2:0]} = {i, kw} hoặc {k, jw}
LOAD_BIAS	1	{0, w[4:0]} = j
STORE_C, INT32	1	{r[4:0], w[4:0]} = {i, j}
STORE_C, INT8	0	{r[4:0], w[2:0]} = {i, jw}
FETCH	—	Không dùng SPM; `o_word_idx` = w (0 … 15)

Chia đoạn và cắt burst

- Lệnh liên tục khi `stride` = 4 · `rwords`: một đoạn dài `rows` · `rwords` word bắt đầu ở base. Ngược lại: `rows` đoạn, đoạn r dài `rwords` word bắt đầu ở base + r · `stride`. Điều kiện liên tục là phép so sánh bằng giữa `i_stride` và {`i_rwords`, 2'b00}
- Trong một đoạn, tại địa chỉ a, burst có min(số word còn lại của đoạn, `MAX_BURST_BEATS`, $1024 - a[11{:}2]$) beat. Burst không bao giờ vượt biên 4 KiB và không bao giờ trải qua khoảng hở giữa hai đoạn
- `o_burst_*` là thanh ghi, hợp lệ từ chu kỳ sau `i_load` và từ chu kỳ sau `i_burst_next`; giá trị kế tiếp được tính từ `cur_addr` và `seg_left` mới trong cùng cạnh clock đó
- `i_burst_next` với burst có nb = `o_burst_beats` beat: nếu `seg_left` > nb thì `cur_addr` ← `cur_addr` + 4 · nb và `seg_left` ← `seg_left` − nb. Nếu hết đoạn và `segs_left` > 1 thì `seg_base` ← `seg_base` + `i_stride`, `cur_addr` ← giá trị mới đó, `seg_left` ← `seg_len`, `segs_left` ← `segs_left` − 1. Nếu hết đoạn cuối thì `o_has_burst` ← 0
- Bộ đếm (r, w) độc lập với burst: `i_word_next` tăng w; khi w = `rwords` − 1 thì w ← 0, r ← r + 1. `o_spm_addr` và `o_word_idx` lấy từ (r, w) theo bảng địa chỉ
- Độ dài đoạn liên tục `rows` · `rwords` ≤ 1024 là một con số đếm (không phải địa chỉ), được tính bằng một bộ nhân tổ hợp 6 × 6 tại `i_load` và nạp vào `seg_len` và `seg_left`

QUYẾT ĐỊNH BASELINE

D-07 (địa chỉ 2D bằng cộng dồn). `seg_base` ← `i_base` lúc start, rồi `seg_base` ← `seg_base` + `stride` sau mỗi hàng (đoạn). Không có bộ nhân trong đường địa chỉ. Lý do: `stride` rộng 16 bit, base 32 bit; cộng dồn chỉ cần một bộ cộng 32 bit thay cho phép nhân 6 × 16 cộng thêm, tốt hơn cho mục tiêu 100 MHz và cho diện tích. Bộ nhân 6 × 6 còn lại chỉ tính độ dài đoạn, nằm ngoài đường địa chỉ.

VÍ DỤ CẮT BURST

LOAD_A với M = 32, K = 32 liên tục (`lda` = 32): một đoạn 256 word; với `MAX_BURST_BEATS` = 16 và base căn 4 KiB: 16 burst 16 beat. Cùng lệnh với `lda` = 48: 32 đoạn 8 word, mỗi đoạn một burst 8 beat (nếu không vượt 4 KiB), khoảng hở 16 byte giữa hai hàng không bị đọc. Base = 0x0FF8, một đoạn 256 word: burst đầu 2 beat (tới biên 4 KiB, vì $1024 - a[11{:}2] = 1024 - 1022 = 2$), sau đó 15 burst 16 beat và một burst 14 beat.

`gemm_dma_wfifo`

Bảng 3.6: Tín hiệu giao tiếp của `gemm_dma_wfifo` (tham số DEPTH = `WFIFO_DEPTH`)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp, bất đồng bộ; chỉ đặt lại con trỏ và bộ đếm.

Bảng 3.6: Tín hiệu giao tiếp của `gemm_dma_wfifo` (tham số DEPTH = `WFIFO_DEPTH`) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_clr`	1	Input	Xóa đồng bộ FIFO về rỗng tại cạnh lên. Do `gemm_dma_wr` cấp: sau lỗi hoặc abort và khi `i_soft_clr`.
`i_push`	1	Input	Ghi `i_din` vào FIFO tại cạnh lên. Bằng `i_spm_rd_rvalid`; không bao giờ phát khi FIFO đầy.
`i_din`	32	Input	Word dữ liệu ghi, bằng `i_spm_rd_rdata`.
`i_pop`	1	Input	Lấy phần tử đầu khỏi FIFO tại cạnh lên. Bằng bắt tay W (`wvalid` ∧ `wready`); không bao giờ phát khi FIFO rỗng.
`o_head`	32	Output	Phần tử đầu FIFO, lấy trực tiếp từ thanh ghi head; chỉ đổi khi có `i_pop` hoặc khi FIFO rỗng nhận phần tử đầu tiên.
`o_empty`	1	Output	FIFO rỗng.
`o_cnt`	3	Output	Số phần tử đang có (0 … `WFIFO_DEPTH`); dùng cho credit đọc SRAM.

3.3 `gemm_dma_rd`: kênh đọc

Trạng thái kênh đọc mã hóa one-hot (4 flip-flop), thứ tự `RD_IDLE`, `RD_ADDR`, `RD_DATA`, `RD_FIN`; reset về RD_IDLE.

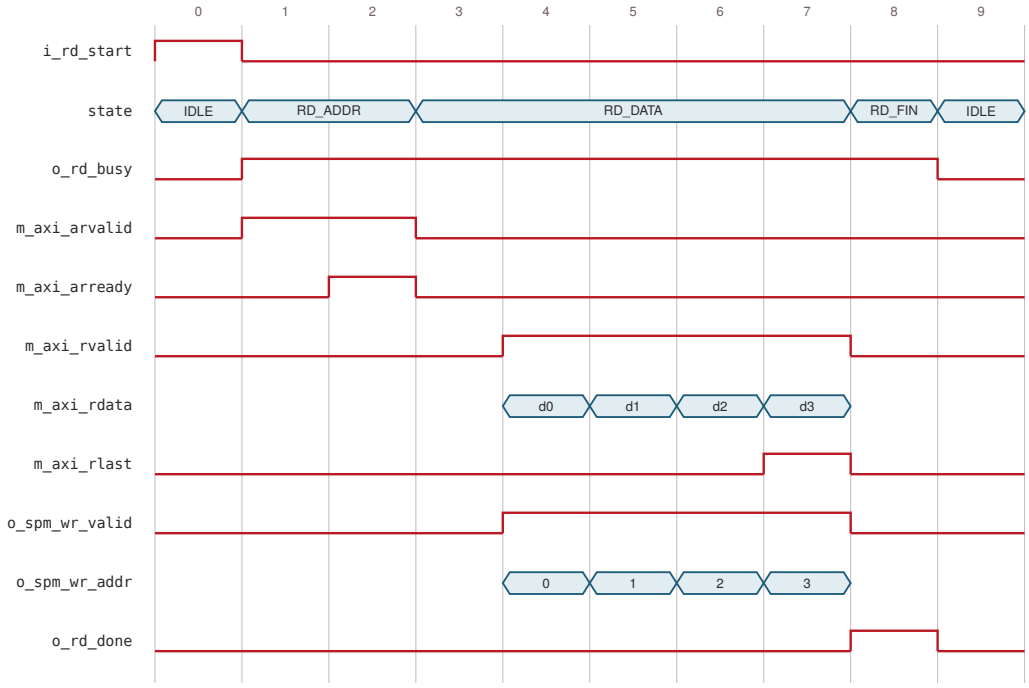
Bảng 3.7: Trạng thái kênh đọc

Trạng thái	Làm gì	Sang
RD_IDLE	`o_rd_busy` = 0. `i_rd_start`: nếu lệnh không hợp lệ (`i_rd_rows` = 0 ∨ `i_rd_rwords` = 0 ∨ `i_rd_stride[1:0]` ≠ 0 ∨ `i_rd_base[1:0]` ≠ 0) thì `err` ← 1, không phát `agen.i_load`. Ngược lại: chốt dst, bank; `agen.i_load`; `err` ← 0; `abort_seen` ← 0	RD_FIN nếu lệnh không hợp lệ; ngược lại RD_ADDR
RD_ADDR	`arvalid` = 1, `araddr` = `o_burst_addr`, `arlen` = beats − 1, giữ tới khi `arready`; `exp` ← beats, `got` ← 0	RD_DATA sau bắt tay AR (`agen.i_burst_next`)
RD_DATA	`rready` = `i_spm_wr_ready` với mọi dst, kể cả dst = 3 (một công thức duy nhất; vì `i_spm_wr_ready` luôn bằng 1 nên thực tế `rready` = 1 suốt RD_DATA). Mỗi beat: `got++`; nếu `got` ≤ `exp`: ghi SRAM hoặc xuất descriptor, `agen.i_word_next`; `rresp` ≠ 0 → `err` ← 1. Xử lý `rlast` theo các quy tắc bên dưới. Burst kết thúc ở beat đầu tiên có `rlast` = 1	Khi burst kết thúc: RD_ADDR nếu `o_has_burst` ∧ ¬`err` ∧ ¬`abort_seen`; ngược lại RD_FIN
RD_FIN	Phát `o_rd_error` nếu `err`, ngược lại `o_rd_done`	RD_IDLE

Kênh đọc

- Tối đa 1 burst outstanding: AR kế tiếp chỉ được phát sau beat cuối của burst trước. Burst đầu tiên của một lệnh hợp lệ được cam kết ngay từ `i_rd_start`
- `rlast` = 1 ở beat có `got` < `exp`: lỗi, burst kết thúc. Beat có `got` = `exp` mà `rlast` = 0: lỗi, tiếp tục nhận (`rready` = 1) và bỏ mọi beat dư (không ghi SRAM, không tăng `agen.i_word_next`) tới khi `rlast` = 1
- Lỗi hoặc abort: không phát AR mới; burst đã phát AR được nhận hết. Abort không đổi kết quả: lệnh bị abort kết thúc bằng done nếu không có lỗi. Abort riêng một mình không phải lỗi
- Dữ liệu beat lỗi vẫn được ghi vào SRAM; FSM không dùng kết quả của lệnh lỗi
- Lệnh có tham số không hợp lệ: `o_rd_busy` = 1 và `o_rd_error` = 1 đúng một chu kỳ ngay sau chu kỳ start; `arvalid` và `o_spm_wr_valid` giữ 0

- Thông lượng: mỗi chu kỳ một beat khi slave không chèn bọt; giữa hai burst có đúng 1 chu kỳ RD_ADDR cộng độ trễ AR → R của slave



Hình 3.1: Lệnh đọc một burst 4 beat vào buffer A (`arready` trễ 1 chu kỳ, slave để trống 1 chu kỳ sau chu kỳ bắt tay AR rồi mới trả beat đầu, `rready` = 1). `arvalid` lên ở chu kỳ 1, bắt tay AR ở cuối chu kỳ 2; 4 beat R ở chu kỳ 4 … 7, `rlast` ở beat cuối; mỗi beat được ghi SRAM ngay trong chu kỳ nhận (hiệu lực ở cạnh clock cuối chu kỳ); `o_rd_done` ở chu kỳ 8 (trạng thái RD_FIN)

3.4 `gemm_dma_wr`: kênh ghi

Trạng thái kênh ghi mã hóa one-hot (5 flip-flop), thứ tự `WR_IDLE`, `WR_NEXT`, `WR_BURST`, `WR_RESP`, `WR_FIN`; reset về WR_IDLE.

Bảng 3.8: Trạng thái kênh ghi

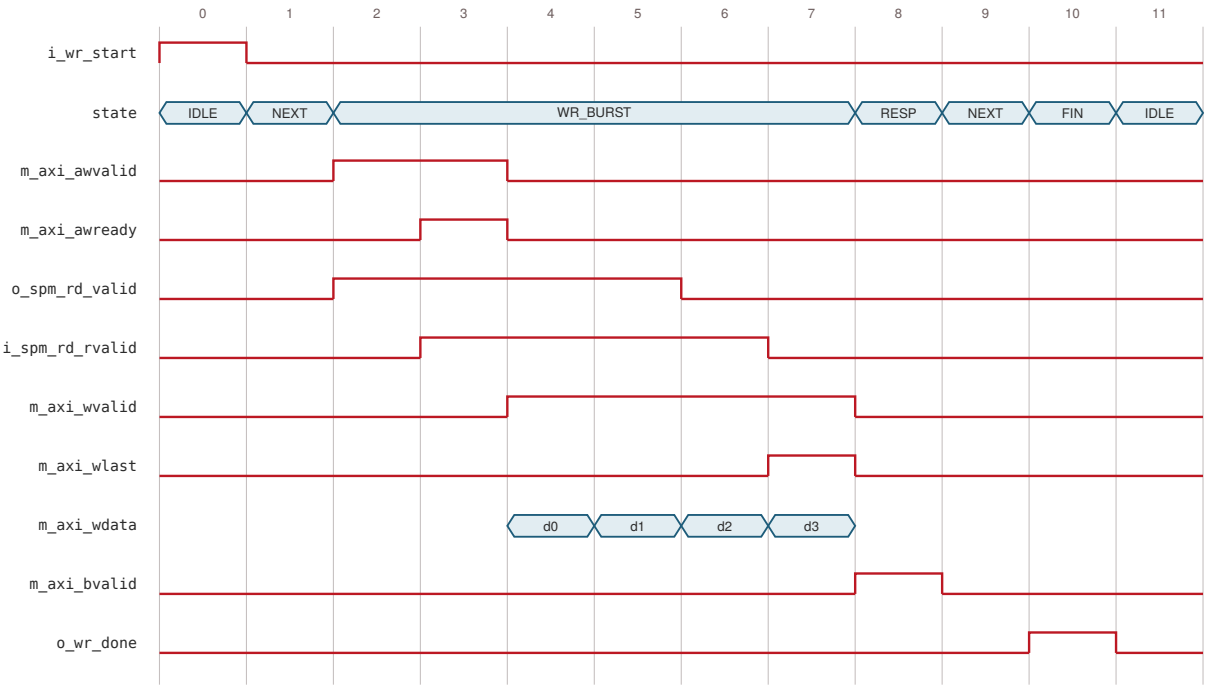
Trạng thái	Làm gì	Sang
WR_IDLE	`i_wr_start`: nếu lệnh không hợp lệ (`i_wr_rows` = 0 ∨ `i_wr_rwords` = 0 ∨ `i_wr_stride[1:0]` ≠ 0 ∨ `i_wr_base[1:0]` ≠ 0) thì `err` ← 1, không phát `agen.i_load`. Ngược lại: chốt fmt, bank; `agen.i_load` với `i_wide` = (fmt = 0); `err`, `abort_seen` ← 0	WR_FIN nếu lệnh không hợp lệ; ngược lại WR_NEXT
WR_NEXT	1 chu kỳ: nếu `err` ∨ `abort_seen` ∨ ¬`o_has_burst` thì sang WR_FIN. Ngược lại nạp `awaddr` ← `o_burst_addr`, `awlen` ← beats − 1, `aw_pend` ← 1, `w_left` ← beats, `rd_left` ← beats; `agen.i_burst_next`	WR_BURST (hoặc WR_FIN như trên)
WR_BURST	Ba việc song song: (1) `awvalid` = `aw_pend` tới khi `awready`; (2) đọc SRAM: `o_spm_rd_valid` = (`rd_left` > 0) ∧ (`fifo_cnt` + `inflight` < `WFIFO_DEPTH`); mỗi yêu cầu được nhận: `rd_left--`, `agen.i_word_next`; (3) `wvalid` = FIFO khác rỗng ∧ `w_left` > 0, `wdata` = đầu FIFO, `wlast` = (`w_left` = 1); mỗi bắt tay W: lấy khỏi FIFO, `w_left--`	WR_RESP khi AW xong và `w_left` = 0
WR_RESP	Chờ `bvalid` (`bready` = 1); `bresp` ≠ 0 → `err` ← 1	WR_NEXT
WR_FIN	Phát `o_wr_error` nếu `err`, ngược lại `o_wr_done`; nếu `err` ∨ `abort_seen` thì xóa FIFO (`gemm_dma_wfifo.i_clr`)	WR_IDLE

QUYẾT ĐỊNH BASELINE

D-08 (credit theo tham số). Điều kiện đọc SRAM của kênh ghi dùng tham số, không dùng hằng 4: `o_spm_rd_valid` = (`rd_left` > 0) && (`fifo_cnt` + `inflight` < `WFIFO_DEPTH`), với `WFIFO_DEPTH` = 4 (localparam). Lý do: tránh hằng số lặp; đổi độ sâu FIFO không làm sai điều kiện. `inflight` = 1 khi có yêu cầu đọc SRAM được nhận ở chu kỳ trước; vì SRAM trả đúng 1 chu kỳ sau nên `inflight` chỉ rộng 1 bit.

Kênh ghi

- `wvalid` không chờ `awready`. Một khi `wvalid` = 1 thì giữ tới bắt tay, vì đầu FIFO chỉ đổi khi bắt tay. `wvalid` được phép về 0 giữa hai beat khi FIFO rỗng
- Burst đã vào WR_BURST là đã cam kết: mọi beat W của nó và phản hồi B đều được hoàn tất, kể cả khi `abort_seen` = 1. Abort chỉ được xét ở WR_NEXT
- Credit: `i_spm_rd_rvalid` đẩy `i_spm_rd_rdata` vào FIFO. FIFO không bao giờ tràn (assertion). Đọc SRAM của burst sau chỉ bắt đầu sau WR_NEXT, nên không có dữ liệu của hai burst cùng nằm trong đường ống
- Tối đa 1 burst ghi outstanding: AW của burst sau chỉ phát sau khi nhận B của burst trước
- Lỗi B (`bresp` ≠ OKAY) làm lệnh kết thúc bằng `o_wr_error` sau khi burst hiện tại đã hoàn tất; không phát burst mới. Lệnh có tham số không hợp lệ kết thúc bằng `o_wr_error` ngay chu kỳ sau start, không có `awvalid` và không có yêu cầu đọc SRAM
- Độ trễ: `i_wr_start` ở chu kỳ t → `awvalid` và yêu cầu đọc SRAM đầu tiên ở t + 2; beat W đầu tiên sớm nhất ở t + 4 (yêu cầu đọc ở t + 2, dữ liệu về FIFO ở t + 3, đầu FIFO có thanh ghi hiện ra ở t + 4); sau đó một beat mỗi chu kỳ khi `wready` = 1



Hình 3.2: Lệnh ghi một burst 4 beat (`awready` trễ 1 chu kỳ, `wready` = 1, `bresp` = OKAY). `i_wr_start` ở chu kỳ t = 0; `awvalid` và yêu cầu đọc SRAM đầu tiên ở t + 2; beat W đầu ở t + 4; `wlast` ở beat thứ 4 (chu kỳ 7); `bvalid` ở chu kỳ 8; WR_NEXT quyết định hết burst ở chu kỳ 9; `o_wr_done` ở chu kỳ 10 (WR_FIN). `o_wr_busy` = 1 từ chu kỳ 1 đến hết chu kỳ 10

3.5 Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0: hai kênh về IDLE; mọi `valid` AXI ra, `o_*_busy`, xung done/error, `o_spm_wr_valid`, `o_spm_rd_valid`, `o_desc_valid`, FIFO về rỗng. `m_axi_bready` giữ 1
- `i_soft_clr`: tác dụng như `rst_n` = 0 đối với trạng thái điều khiển (xóa đồng bộ). Main FSM chỉ phát `soft_clr` khi cả hai kênh đã rảnh, nên soft clear không bao giờ cắt ngang một giao dịch AXI và không tạo done hay error

Hiện thực

- SystemVerilog tổng hợp được, không latch. Thanh ghi trạng thái (trạng thái one-hot của hai FSM, `err`, `abort_seen`, `aw_pend`, các bộ đếm, con trỏ FIFO) viết bằng `always_ff @(posedge clk or negedge rst_n)` và có reset; thanh ghi chỉ chứa dữ liệu (`seg_base`, `cur_addr`, `araddr`, `awaddr`, nội dung FIFO) viết bằng `always_ff @(posedge clk)` và không có giá trị reset
- Số beat của burst được tính bằng so sánh ba giới hạn: số word còn lại trong đoạn, `MAX_BURST_BEATS` và số word tới biên 4 KiB. Giới hạn cuối bằng `1024 - a[11:2]`
- `MAX_BURST_BEATS` ngoài 1 … 256 bị chặn lúc elaboration

> **QUYẾT ĐỊNH BASELINE**
> **D-21 (kiểu always_ff).** Flip-flop trạng thái dùng `always_ff @(posedge clk or negedge rst_n)`, trong đó `rst_n` là `rst_n_sync` do top cấp; flip-flop chỉ chứa dữ liệu (địa chỉ, FIFO) dùng `always_ff @(posedge clk)` và không có giá trị reset. `soft_clr` là xóa đồng bộ một chu kỳ trong nhánh `else`, không đi vào chân reset.

Assertion bắt buộc (SVA)

- **AS-01**: `arvalid` $\wedge$ `¬arready` $\rightarrow$ ở chu kỳ sau `arvalid` = 1 và `araddr`, `arlen` không đổi; tương tự AW, W (cả `wdata`, `wlast`)
- **AS-02**: `arvalid` $\rightarrow$ `arlen` < `MAX_BURST_BEATS` $\wedge$ `araddr[11:2]` + `arlen` $\leq$ 1023; tương tự AW
- **AS-03**: không có AR mới khi burst đọc trước chưa có `rlast`; không có AW mới khi chưa nhận B của burst trước
- **AS-04**: số beat W giữa hai lần `wlast` = `awlen` + 1 của burst tương ứng
- **AS-05**: `i_rd_start` $\wedge$ `¬o_rd_busy` $\rightarrow$ đúng một `o_rd_done` hoặc `o_rd_error` sau đó; `o_rd_done` và `o_rd_error` không bao giờ cùng chu kỳ; tương tự kênh ghi
- **AS-06**: sau khi `abort_seen` = 1, không có `arvalid` hay `awvalid` nào bật lên mới
- **AS-07**: FIFO không tràn, không lấy khi rỗng
- **AS-08**: `i_soft_clr` $\rightarrow$ cả hai kênh ở IDLE ở chu kỳ sau
- **AS-09**: `o_spm_rd_valid` $\rightarrow$ `fifo_cnt` + `inflight` < `WFIFO_DEPTH`
- **AS-10**: lệnh có tham số không hợp lệ $\rightarrow$ `o_*_error` ở chu kỳ ngay sau start và không có `arvalid`, `awvalid`, `o_spm_wr_valid`, `o_spm_rd_valid` nào trong suốt lệnh
- **AS-11**: `m_axi_bready` = 1, `m_axi_arsize` = `m_axi_awsize` = 3'b010, `m_axi_arburst` = `m_axi_awburst` = 2'b01, `m_axi_wstrb` = 4'b1111 ở mọi chu kỳ

Kiểm tra chức năng

- Chuỗi địa chỉ DRAM và scratchpad của mọi word khớp công thức base + hàng $\times$ stride + 4 $\times$ word và bảng địa chỉ scratchpad; không burst nào vượt 4 KiB hay trải qua khoảng hở; ba ví dụ cắt burst ở mục 3.2 cho đúng 16 $\times$ 16 beat, 32 $\times$ 8 beat và 2 + 15 $\times$ 16 + 14 beat
- `rlast` sớm: lệnh kết thúc bằng `o_rd_error`, không AR mới; `rlast` muộn: lệnh nhận hết beat tới `rlast`, bỏ beat dư, kết thúc bằng `o_rd_error`; dữ liệu beat lỗi vẫn có trong SRAM

- Abort một mình: lệnh kết thúc bằng done nếu không lỗi; lệnh ghi bị abort chỉ dừng ở biên burst
- FETCH trả đúng 16 word qua `o_desc_*` theo thứ tự chỉ số 0 … 15 và không có `o_spm_wr_valid`
- Độ trễ ghi: start ở t, `awvalid` và yêu cầu đọc SRAM đầu ở t + 2, beat W đầu ở t + 4

Bảng 3.9: Tài nguyên ước tính

Hạng mục	Số lượng
Bộ cộng 32 bit (địa chỉ burst, địa chỉ đoạn)	4 (2 mỗi kênh)
Bộ nhân 6 × 6 (độ dài đoạn liên tục, không thuộc đường địa chỉ)	2 (1 mỗi kênh)
FIFO ghi	4 × 32 bit
Thanh ghi trạng thái và tham số	khoảng 2 × 140 bit

KIỂM CHỨNG BẰNG MÔ HÌNH

(Mô hình nằm ở `gemm-accel/model/`, đã chạy lại với baseline ngày 12/11/2026; kết quả đạt.) `gemm_dma_model.py`: (1) 15 000 lệnh ngẫu nhiên (5 loại, stride liên tục hoặc có hở, `MAX_BURST_BEATS` 1 … 256): chuỗi địa chỉ DRAM và scratchpad khớp công thức base + hàng × stride + 4 × word, không burst nào vượt 4 KiB hay trải qua khoảng hở; (2) 1 500 lệnh kênh đọc và 1 500 lệnh kênh ghi với slave ngẫu nhiên, gồm lỗi RESP, `rlast` sớm hoặc trễ, abort: dữ liệu đúng khi không lỗi, không AR sau abort, không burst nào bị bỏ dở.